

The Recursive Complete Minimum-Norm HSS Solver

algorithm, proof of optimality, and numerical experiments

Arjun Sethi-Olowin

October 2026

1 Introduction

These notes describe a direct, ULV-type algorithm that computes the minimum-norm solution $x^* = H^+b$ of $Hx = b$, where H is a wide (or square) hierarchically semiseparable (HSS) matrix of full row rank. The algorithm works one tree level at a time: unitary transformations decouple part of the unknowns of every leaf, those unknowns are solved for directly, and what remains is a smaller HSS system with one level fewer. We prove that the result is exactly H^+b and that the cost is linear in n , and test the algorithm numerically.

- Section 2: notation and the one standing assumption on the representation.
- Section 3: one level of the algorithm (size reduction, decoupling, forced solve, reduced matrix, reconstruction), and why the slack condition of the fixed-width formulation is not needed.
- Section 4: proof that the algorithm returns H^+b : four lemmas, one level, all levels.
- Section 5: complexity, with measured times.
- Section 6: the algorithm in pseudocode.
- Section 7: numerical experiments: test matrices, correctness against dense solutions, accuracy as the condition number grows, accuracy at large n .
- Section 8: open problems.

2 Setup and Notation

We consider a matrix $H \in \mathbb{C}^{m \times n}$, $m \leq n$, of full row rank, given in HSS form, and want

$$x^* = \arg \min_{Hx=b} \|x\|_2 = H^*(HH^*)^{-1}b = H^+b.$$

For $m = n$ this is the solution $H^{-1}b$; the algorithm covers both cases.

Tree and generators. $\mathcal{T}$ is a perfect binary tree of depth L . Node τ owns a row index set I_τ and a column index set J_τ , and the children of a node partition its index sets. For a *leaf* τ ,

$$D_\tau = H(I_\tau, J_\tau) \in \mathbb{C}^{n_\tau \times p_\tau}, \quad U_\tau \in \mathbb{C}^{n_\tau \times k_\tau}, \quad V_\tau \in \mathbb{C}^{p_\tau \times \ell_\tau} :$$

a leaf has n_τ rows and p_τ columns, k_τ is the rank of its row basis and ℓ_τ the rank of its column basis. (So n_τ counts the *rows* of a leaf, while n counts the *columns* of H .) A non-leaf, non-root

node π with children α, β stores translation matrices $R_\pi \in \mathbb{C}^{(k_\alpha+k_\beta) \times k_\pi}$ and $W_\pi \in \mathbb{C}^{(\ell_\alpha+\ell_\beta) \times \ell_\pi}$, and the nested bases are

$$\mathbf{U}_\tau = U_\tau, \quad \mathbf{V}_\tau = V_\tau \text{ } (\tau \text{ a leaf}), \quad \mathbf{U}_\pi = \begin{bmatrix} \mathbf{U}_\alpha & \\ & \mathbf{U}_\beta \end{bmatrix} R_\pi, \quad \mathbf{V}_\pi = \begin{bmatrix} \mathbf{V}_\alpha & \\ & \mathbf{V}_\beta \end{bmatrix} W_\pi.$$

For siblings α, β (children of the same node),

$$H(I_\alpha, J_\beta) = \mathbf{U}_\alpha B_{\alpha\beta} \mathbf{V}_\beta^*, \quad B_{\alpha\beta} \in \mathbb{C}^{k_\alpha \times \ell_\beta}. \quad (1)$$

Unrolling the nested bases, the part of leaf τ 's block row outside its diagonal block is $H(I_\tau, J \setminus J_\tau) = U_\tau G_\tau$, and the part of its block column outside the diagonal block is $H(I \setminus I_\tau, J_\tau) = F_\tau V_\tau^*$, for some matrices G_τ, F_τ . Consequently, a unitary transformation of the rows of leaf τ acts on its whole block row through U_τ and D_τ alone, and a unitary transformation of its columns acts on its whole block column through V_τ and D_τ alone. This is what makes the leaf-local operations below global.

Proper representation. We assume that no basis is wider than the block it spans: $k_\tau \leq n_\tau$ for every leaf, and $k_\pi \leq k_\alpha + k_\beta$ for every non-leaf π with children α, β . This is a property of the representation, not of H : any representation can be brought to this form without changing H (replace a wide basis by an orthonormal basis of its range and absorb the remaining factor into the parent's translation and into B), and nested-basis constructions produce it directly. The algorithm uses it only so that the number of decoupled rows $t_\tau = n_\tau - k_\tau$ in Section 3.2 is nonnegative.

Right compressions and QL factorizations. For $M \in \mathbb{C}^{a \times p}$, $a \leq p$, a *right compression* is a unitary $Q \in \mathbb{C}^{p \times p}$ with $MQ = [L, 0]$, $L \in \mathbb{C}^{a \times a}$ lower triangular; it comes from a QR factorization $M^* = Q \begin{bmatrix} R \\ 0 \end{bmatrix}$, $L = R^*$. A *QL factorization* of a tall $U \in \mathbb{C}^{n \times k}$ ($k \leq n$) is $U = P \begin{bmatrix} 0 \\ \hat{U} \end{bmatrix}$ with P unitary and $\hat{U} \in \mathbb{C}^{k \times k}$ lower triangular.

3 One Level of the Algorithm

The algorithm is a minimum-norm variant of the ULV elimination of [1]. One level processes every leaf τ independently and then forms a reduced HSS system with one level fewer, which is solved recursively. We list the operations for a leaf τ with parent π and sibling σ .

3.1 Size reduction

If $\ell_\tau + n_\tau < p_\tau$, compute a unitary Ω_τ (from a QR factorization of the $p_\tau \times (\ell_\tau + n_\tau)$ matrix $[V_\tau, D_\tau^*]$) with

$$\begin{bmatrix} V_\tau^* \\ D_\tau \end{bmatrix} \Omega_\tau = \begin{bmatrix} \tilde{V}_\tau^* & 0 \\ \tilde{D}_\tau & 0 \end{bmatrix}, \quad \tilde{V}_\tau^* \in \mathbb{C}^{\ell_\tau \times p'_\tau}, \quad \tilde{D}_\tau \in \mathbb{C}^{n_\tau \times p'_\tau}, \quad (2)$$

where $p'_\tau = \ell_\tau + n_\tau$. Otherwise there is nothing to remove: $\Omega_\tau = I$ and $p'_\tau = p_\tau$. In both cases $p'_\tau = \min(p_\tau, \ell_\tau + n_\tau)$. Every row of $H(:, J_\tau)$ is a combination of the rows of $[V_\tau^*; D_\tau]$ (Section 2), so the last $p_\tau - p'_\tau$ columns of $H(:, J_\tau)\Omega_\tau$ vanish in every block row. Dropping them gives $\tilde{H}$: the same tree and generators, except $D_\tau \rightarrow \tilde{D}_\tau$ and $V_\tau \rightarrow \tilde{V}_\tau$.

Assumption 1 (slack). *Every leaf satisfies $\ell_\tau + n_\tau \leq p_\tau$.*

Under Assumption 1 the size reduction removes $p_\tau - \ell_\tau - n_\tau \geq 0$ columns from every leaf and leaves exactly $p'_\tau = \ell_\tau + n_\tau$, so all block sizes are known in advance; a formulation with this fixed width needs the assumption. The algorithm itself does not, for two reasons.

1. The size reduction only discards columns along which H vanishes identically. When $\ell_\tau + n_\tau \geq p_\tau$ there may be none, and $\Omega_\tau = I$. Lemma 4.1 below, which justifies the step, holds with an empty set of discarded columns.
2. After the size reduction, the sizes enter at exactly one point: the forced block L_τ of Section 3.2 must be square, which requires $t_\tau \leq p'_\tau$. This follows from full row rank alone (Lemma 4.4): the t_τ decoupled rows of leaf τ are nonzero only in τ 's p'_τ columns, and the rows of a full-row-rank matrix are linearly independent.

Without the assumption, a reduced leaf has $f_\tau = p'_\tau - t_\tau$ free columns, which can be fewer than its k_τ coupled rows; reduced leaves can then be square or tall. Only the reduced matrix as a whole has to have full row rank, and Lemma 4.4 shows that it does.

3.2 Decoupling

Step 1 (row compression). QL-factor $U_\tau = P_\tau \begin{bmatrix} 0 \\ \tilde{U}_\tau \end{bmatrix}$ and multiply τ 's rows (of $\tilde{D}_\tau$ and of b) by P_τ^* . With $t_\tau := n_\tau - k_\tau$, the top t_τ rows of $P_\tau^* U_\tau$ are zero, so the top t_τ rows of τ 's block row vanish outside J_τ .

Step 2 (column decoupling). Right-compress the top rows, $(P_\tau^* \tilde{D}_\tau)(1:t_\tau, :) Q_\tau = [L_\tau, 0]$, with $L_\tau \in \mathbb{C}^{t_\tau \times t_\tau}$ lower triangular and $Q_\tau \in \mathbb{C}^{p'_\tau \times p'_\tau}$ unitary (this needs $t_\tau \leq p'_\tau$, which holds by Lemma 4.4), and apply Q_τ to all of τ 's columns:

$$\hat{D}_\tau := P_\tau^* \tilde{D}_\tau Q_\tau = \begin{bmatrix} L_\tau & 0 \\ C_\tau & E_\tau \end{bmatrix}, \quad \hat{V}_\tau^* := \tilde{V}_\tau^* Q_\tau = \begin{bmatrix} \hat{V}_{\tau,1}^* & \hat{V}_{\tau,2}^* \end{bmatrix}, \quad (3)$$

with $C_\tau \in \mathbb{C}^{k_\tau \times t_\tau}$ and $E_\tau \in \mathbb{C}^{k_\tau \times f_\tau}$, $f_\tau = p'_\tau - t_\tau$. Write $\hat{H} = P^* \tilde{H} Q$ and $\hat{b} = P^* b$, with $P = \text{diag}(P_\tau)$ and $Q = \text{diag}(Q_\tau)$.

Zero pattern. Split the rows of each leaf into the top t_τ (*decoupled* rows, set $\mathcal{R}_1$) and the bottom k_τ ($\mathcal{R}_2$), and its columns into the first t_τ (*forced* columns, $\mathcal{C}_1$) and the remaining f_τ (*free* columns, $\mathcal{C}_2$). Then

$$\hat{H} = \begin{bmatrix} \hat{H}(\mathcal{R}_1, \mathcal{C}_1) & \hat{H}(\mathcal{R}_1, \mathcal{C}_2) \\ \hat{H}(\mathcal{R}_2, \mathcal{C}_1) & \hat{H}(\mathcal{R}_2, \mathcal{C}_2) \end{bmatrix} = \begin{bmatrix} \mathbf{L} & 0 \\ M & \bar{H} \end{bmatrix}, \quad \mathbf{L} = \text{diag}(L_\tau). \quad (4)$$

$\hat{H}(\mathcal{R}_1, \mathcal{C}_1)$ is block diagonal because a decoupled row of τ is zero in every other leaf's columns (Step 1), and $\hat{H}(\mathcal{R}_1, \mathcal{C}_2) = 0$ because, in addition, it is zero in τ 's own free columns (Step 2).

3.3 Forced solve and the reduced right-hand side

Write $\hat{x}_\tau = [X_\tau^{(1)}; X_\tau^{(2)}]$ (forced; free). The decoupled rows read $L_\tau X_\tau^{(1)} = \hat{b}_{\tau,1}$, solved by forward substitution. The remaining rows read $\bar{H} X^{(2)} = \bar{b}$ with

$$\bar{b} = \hat{b}(\mathcal{R}_2) - M X^{(1)} = \left(\hat{b} - \hat{H} \begin{bmatrix} X^{(1)} \\ 0 \end{bmatrix} \right)(\mathcal{R}_2). \quad (5)$$

For the bottom rows of leaf τ this is $\bar{b}_\tau = \hat{b}_{\tau,2} - C_\tau X_\tau^{(1)} - \hat{U}_\tau (B_{\tau\sigma} \hat{V}_{\sigma,1}^* X_\sigma^{(1)} + g_\tau)$, where g_τ collects the contributions of the non-sibling leaves through the nested bases: the forced unknowns of every leaf reach τ 's rows. The second form of (5) computes $\bar{b}$ with one HSS matrix–vector product.

3.4 The reduced matrix is HSS with one level fewer

Discard the decoupled rows and the forced columns. For a parent π of leaves a, c the new leaf is

$$\bar{D}_\pi = \begin{bmatrix} E_a & \hat{U}_a B_{ac} \hat{V}_{c,2}^* \\ \hat{U}_c B_{ca} \hat{V}_{a,2}^* & E_c \end{bmatrix}, \quad \bar{U}_\pi = \begin{bmatrix} \hat{U}_a & \\ & \hat{U}_c \end{bmatrix} R_\pi, \quad \bar{V}_\pi = \begin{bmatrix} \hat{V}_{a,2} & \\ & \hat{V}_{c,2} \end{bmatrix} W_\pi, \quad (6)$$

of size $(k_a + k_c) \times (f_a + f_c)$, and every generator above π is unchanged. Its row basis $\bar{U}_\pi$ has $k_\pi \leq k_a + k_c$ columns, so the reduced representation is again proper.

3.5 Reconstruction

Given $\bar{x} = [X_\tau^{(2)}]_\tau$ from the recursive call,

$$x_\tau = \Omega_\tau \begin{bmatrix} Q_\tau \begin{bmatrix} X_\tau^{(1)} \\ X_\tau^{(2)} \end{bmatrix} \\ 0 \end{bmatrix}$$

(without the zero block when $\Omega_\tau = I$). At the root the matrix is a single dense block D of full row rank, and the algorithm returns $D^+ b$, computed by applying the factors of an SVD of D to b (for square D , $D^{-1}b$ from an LU factorization). Forming D^+ explicitly and multiplying would not be backward stable [4].

4 Proof of Optimality

4.1 Four lemmas

Lemma 4.1 (right unitary substitution). *Let Ω be unitary with $A\Omega = [\tilde{A}, 0]$, where the zero block may be empty. If $\tilde{x}^*$ is the minimum-norm solution of $\tilde{A}\tilde{x} = b$, then $x^* = \Omega \begin{bmatrix} \tilde{x}^* \\ 0 \end{bmatrix}$ is the minimum-norm solution of $Ax = b$.*

Proof. Every x can be written uniquely as $x = \Omega[\tilde{x}; w]$, and since Ω is unitary, $\|x\|^2 = \|\tilde{x}\|^2 + \|w\|^2$. Because $Ax = [\tilde{A}, 0][\tilde{x}; w] = \tilde{A}\tilde{x}$, the vector x solves $Ax = b$ exactly when $\tilde{x}$ solves $\tilde{A}\tilde{x} = b$, whatever w is. Among the solutions, $\|x\|$ is therefore smallest when $w = 0$ and $\tilde{x}$ is the solution of least norm, $\tilde{x} = \tilde{x}^*$. $\square$

Lemma 4.2 (two-sided unitary substitution). *Let P, Q be unitary, $\hat{A} = P^* A Q$ and $\hat{b} = P^* b$. If $\hat{x}^*$ is the minimum-norm solution of $\hat{A}\hat{x} = \hat{b}$, then $x^* = Q\hat{x}^*$ is the minimum-norm solution of $Ax = b$.*

Proof. Since P^* is invertible, $Ax = b$ holds if and only if $P^* A Q(Q^* x) = P^* b$, that is, $\hat{A}\hat{x} = \hat{b}$ with $\hat{x} = Q^* x$. The map $x \mapsto Q^* x$ is therefore a bijection between the two solution sets, and it preserves the norm, so it maps the minimizer of one problem to the minimizer of the other. $\square$

Lemma 4.3 (forced block). *Let $A = \begin{bmatrix} \mathbf{L} & 0 \\ M & \bar{A} \end{bmatrix}$ with $\mathbf{L}$ square and nonsingular, and $b = [b_1; b_2]$. Then the minimum-norm solution of $Ax = b$ is $x^* = [\mathbf{L}^{-1}b_1; \bar{x}^*]$, where $\bar{x}^*$ is the minimum-norm solution of $\bar{A}\bar{x} = b_2 - M\mathbf{L}^{-1}b_1$.*

Proof. The first block row reads $\mathbf{L}x_1 = b_1$, so every solution has $x_1 = \mathbf{L}^{-1}b_1$. Substituting this into the second block row, $x = [x_1; x_2]$ solves $Ax = b$ if and only if $x_1 = \mathbf{L}^{-1}b_1$ and $\bar{A}x_2 = b_2 - M\mathbf{L}^{-1}b_1$. Since $\|x\|^2 = \|\mathbf{L}^{-1}b_1\|^2 + \|x_2\|^2$ and the first term is the same for every solution, $\|x\|$ is minimized by taking x_2 of least norm among the solutions of the reduced system. $\square$

Lemma 4.4 (full row rank is inherited). *Let H have full row rank and apply one level (Sections 3.1–3.4). Then (i) $t_\tau \leq p'_\tau$ for every leaf, so L_τ is square; (ii) every L_τ is nonsingular; (iii) $\bar{H}$ has full row rank.*

Proof. The size reduction and the two unitary transformations do not change the rank: $H\Omega = [\tilde{H}, 0]$ up to a column permutation, and $\hat{H} = P^*\tilde{H}Q$. Hence $\text{rank } \hat{H} = \text{rank } H = m$, and the m rows of $\hat{H}$ are linearly independent. Consider the t_τ decoupled rows of leaf τ . By Step 1 they are zero outside τ 's p'_τ columns, and t_τ linearly independent vectors with only p'_τ nonzero coordinates require $t_\tau \leq p'_\tau$; this is (i). By Step 2 these rows equal $[L_\tau, 0]$ on τ 's columns, so their independence means that L_τ has rank t_τ ; this is (ii). For (iii), (4) factors as

$$\begin{bmatrix} \mathbf{L} & 0 \\ M & \bar{H} \end{bmatrix} = \begin{bmatrix} I & 0 \\ M\mathbf{L}^{-1} & I \end{bmatrix} \begin{bmatrix} \mathbf{L} & 0 \\ 0 & \bar{H} \end{bmatrix},$$

and the first factor is invertible, so $m = \text{rank } \mathbf{L} + \text{rank } \bar{H} = \sum_\tau t_\tau + \text{rank } \bar{H}$. Since $\bar{H}$ has $m - \sum_\tau t_\tau$ rows, it has full row rank. $\square$

Note that no part of the proof uses Assumption 1: the inequality $t_\tau \leq p'_\tau$ is a consequence of full row rank, not a hypothesis.

4.2 One level

Theorem 4.5 (one level). *Let H (depth $L \geq 1$) have full row rank, and let $\bar{x}^*$ be the minimum-norm solution of $\bar{H}\bar{x} = \bar{b}$. Then the vector x assembled from $\bar{x}^*$ as in Section 3.5 is the minimum-norm solution of $Hx = b$.*

Proof. Each step of Section 3 replaces the minimum-norm problem by an equivalent one, by one of the lemmas.

1. *Size reduction.* With $\Omega = \text{diag}(\Omega_\tau)$, $H\Omega = [\tilde{H}, 0]$ up to a column permutation. By Lemma 4.1, $x^* = \Omega[\tilde{x}^*; 0]$, where $\tilde{x}^*$ is the minimum-norm solution of $\tilde{H}\tilde{x} = b$.
2. *Row compression and column decoupling.* With $P = \text{diag}(P_\tau)$ and $Q = \text{diag}(Q_\tau)$, Lemma 4.2 gives $\tilde{x}^* = Q\hat{x}^*$, where $\hat{x}^*$ is the minimum-norm solution of $\hat{H}\hat{x} = \hat{b}$.
3. *Forced solve.* By (4) and Lemma 4.4(ii), $\hat{H}$ has the form of Lemma 4.3 with $\mathbf{L}$ nonsingular, so $\hat{x}^* = [X^{(1)}; \bar{x}^*]$, where $X^{(1)} = \mathbf{L}^{-1}\hat{b}(\mathcal{R}_1)$ and $\bar{x}^*$ is the minimum-norm solution of $\bar{H}\bar{x} = \bar{b}$ with $\bar{b}$ as in (5).

Combining the three steps, $x^* = \Omega[Q[X^{(1)}; \bar{x}^*]; 0]$, which is the reconstruction of Section 3.5 applied leaf by leaf. $\square$

4.3 All levels

Theorem 4.6 (recursive algorithm). *For every HSS matrix H of full row rank, of any depth, the algorithm (Algorithm 1) returns H^+b in exact arithmetic.*

Proof. Induction on the depth L . If $L = 0$, $H = D$ is a dense matrix of full row rank and the algorithm returns $D^+b = H^+b$. Let $L \geq 1$ and assume the claim for depth $L - 1$. By Section 3.4 and Lemma 4.4(iii), $\bar{H}$ is an HSS matrix of depth $L - 1$ with full row rank, so by the induction hypothesis the recursive call returns $\bar{x}^* = \bar{H}^+\bar{b}$. By Theorem 4.5, the reconstructed vector is H^+b . $\square$

4.4 Scope of the result

The theorems are statements about the HSS matrix H in exact arithmetic. They carry over to the original matrix as far as the HSS representation is accurate: if $\|A - H\| \leq \varepsilon\|A\|$, then H^+b differs from A^+b by at most about $2\kappa(A)\varepsilon$ in relative terms, to first order (the condition number of the minimum-norm problem is about 2κ , not κ^2 [2]). Floating-point stability is not proved here. Section 7.4 shows that the computed solution is as accurate as the dense QR solution for κ up to 1.8×10^{13} , the largest value tested.

5 Complexity

Let the leaves have n_0 rows and p_0 columns, and let all ranks be at most r . One leaf costs $O(p_0(n_0 + r)^2)$: the QR factorization of the $p_0 \times (\ell + n_0)$ matrix $[V_\tau, D_\tau^*]$ in the size reduction dominates, and the QL factorization, the products with P_τ and Q_τ and the triangular solve are of the same or lower order. There are about m/n_0 leaves. Every reduced node has at most $2r$ rows and $4r$ columns, so it costs $O(r^3)$, and there are fewer than m/n_0 of them. The right-hand-side update (5) is one HSS product per level, $O(m(n_0 + r))$ at the leaf level and less above. With $mp_0/n_0 = n$,

$$\text{time} = O\left(n(n_0 + r)^2 + \frac{m}{n_0}r^3\right) = O(nr^2) \quad \text{for } n_0 \sim r, \quad \text{memory} = O(n(n_0 + r)).$$

At a fixed leaf size the cost is linear in n . Once the factors are computed, a solve with a new right-hand side costs $O(n(n_0 + r))$, i.e. $O(nr)$ for $n_0 \sim r$: triangular solves, small unitary products and one HSS product per level. By comparison, the dense minimum-norm solve costs $O(m^2n)$, and conjugate gradients on HH^* (CGNE) costs two HSS products per iteration with an iteration count that grows with $\kappa(H)$.

Figure 1 shows measured times for random HSS matrices with $m = n/2$ (family F2 of Section 7.2) and for families F1–F5, up to $n = 2^{17}$, with leaves of 200–400 rows. The time of the first solve, which computes the factors, grows linearly in n for every family (fitted exponents 0.97–1.05 over the last four sizes), and so does the time of a repeat solve with the stored factors (exponents 0.97–1.04). At the largest size a repeat solve is 25–46 times faster than the first solve and costs as much as 2–3 HSS matrix–vector products. The dense QR solve grows like $n^{2.6}$ and is $38\times$ slower than the first ULV solve at $n = 8192$. CGNE needs 79–171 iterations even on these well-conditioned matrices, each with two HSS products; at $n = 32768$ it is $6\times$ slower than the first ULV solve and $150\times$ slower than a repeat solve, and Section 7.4 shows that it stops converging as κ grows. The timings come from `mp_exp_scaling.m`.

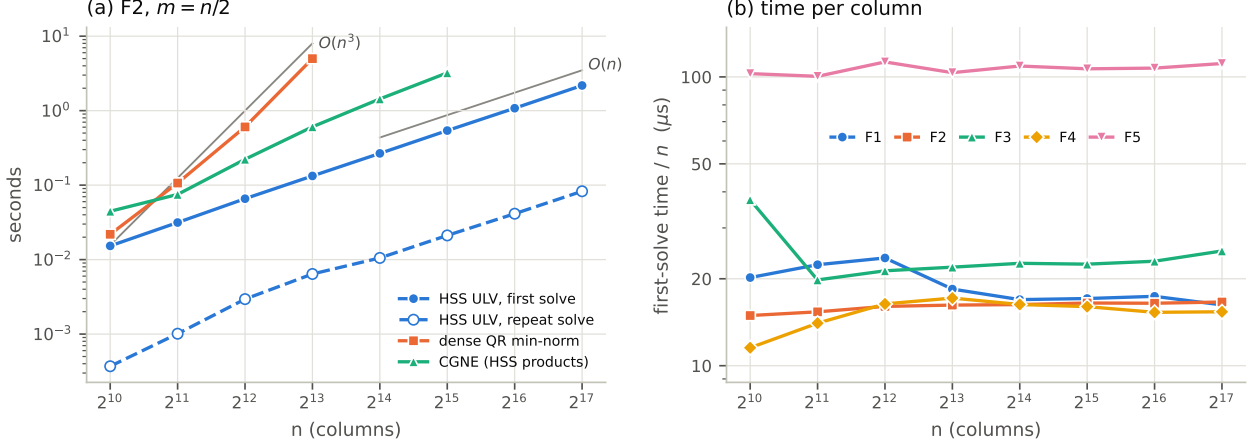


Figure 1: Measured cost. (a) F2, $m = n/2$, rank 10: first solve (factorization and solve), repeat solve with the stored factors, dense QR minimum-norm solve and CGNE (conjugate gradients on HH^* with HSS products, to relative residual 10^{-12}); grey lines $O(n)$ and $O(n^3)$. (b) Time of the first solve divided by n , for F1–F5: a flat line is linear cost (the first F3 point is a single dense block). Times from MATLAB R2025b (Apple silicon Mac, 10 threads); HSS solves are the median of three runs after a warm-up, each first solve starting without stored factors; dense QR and CGNE are single runs.

6 Algorithm Summary

Algorithm 1 Recursive Complete Minimum-Norm HSS Solve, $\mathcal{M}(H, b)$

Require: HSS matrix $H \in \mathbb{C}^{m \times n}$, $m \leq n$, full row rank, proper representation; $b \in \mathbb{C}^m$

- 1: **if** H is a single dense block D **then**
 - 2: **return** D^+b ▷ from an SVD of D ; $D^{-1}b$ if square
 - 3: **end if**
 - 4: **for all** leaves τ **do**
 - 5: **if** $\ell_\tau + n_\tau < p_\tau$: $[V_\tau^*; D_\tau] \Omega_\tau = [\tilde{V}_\tau^*, 0; \tilde{D}_\tau, 0]$; **else** $\Omega_\tau = I$ ▷ size reduction
 - 6: $P_\tau^* U_\tau = [0; \tilde{U}_\tau]$; $\tilde{D}_\tau \leftarrow P_\tau^* \tilde{D}_\tau$; $\hat{b}_\tau = P_\tau^* b_\tau$ ▷ row compression
 - 7: $\tilde{D}_\tau(1:t_\tau, :) Q_\tau = [L_\tau, 0]$; $\hat{D}_\tau = \tilde{D}_\tau Q_\tau$; $\hat{V}_\tau^* = \tilde{V}_\tau^* Q_\tau$ ▷ column decoupling
 - 8: $X_\tau^{(1)} = L_\tau^{-1} \hat{b}_\tau(1:t_\tau)$ ▷ forced solve
 - 9: **end for**
 - 10: $\bar{b} \leftarrow (\hat{b} - \hat{H}[X^{(1)}; 0])(\mathcal{R}_2)$ ▷ one HSS product, (5)
 - 11: $\bar{H} \leftarrow$ discard decoupled rows and forced columns, merge sibling leaves (6)
 - 12: $\bar{x} \leftarrow \mathcal{M}(\bar{H}, \bar{b})$ ▷ one level fewer
 - 13: **return** $x_\tau = \Omega_\tau [Q_\tau [X_\tau^{(1)}; \bar{x}_\tau]; 0]$ for every leaf
-

7 Numerical Experiments

7.1 Setup

Each subsection names the MATLAB test file that produces its results. The accuracy tests at moderate size use leaves of 16–64 rows, so that every tree has 3–7 levels; the timing and large- n runs use leaves of 200–400 rows. All results were computed with MATLAB R2025b (Apple silicon

Mac, 10 threads).

With x the computed solution, we report:

- the residual $\|Hx - b\|/\|b\|$;
- the *solver error* $\|x - x_{\text{ref}}\|/\|x_{\text{ref}}\|$, where x_{ref} is the dense minimum-norm solution of the HSS matrix itself, computed from a QR factorization $H^* = QR$ as $x_{\text{ref}} = Q(R^{-*}b)$. This measures the solver alone, without the compression error;
- the error against the minimum-norm solution of the original matrix A , which adds the compression error;
- the null-space fraction $\|P_{\mathcal{N}(H)}x\|/\|x\|$, which is 0 for an exact minimum-norm solution;
- for large n , where no dense matrix can be formed, the error against a *manufactured* solution: draw z , set $x_\star = H^*z$ and $b = Hx_\star$. Since $x_\star \in \text{Range}(H^*)$ and $Hx_\star = b$, $x_\star$ is H^+b , and $\|x - x_\star\|/\|x_\star\|$ needs only HSS products.

7.2 Test matrices

Table 1 defines the families. F1 and F2 are given directly by their generators, so they are exactly HSS; F3–F6 are compressed from their entries with nested interpolative decompositions. Large- n cases never form a dense matrix.

	definition	HSS form	κ
F1	$A = XY + \text{blkdiag}(E_\tau)$, $X \in \mathbb{R}^{m \times k}$, $Y \in \mathbb{R}^{k \times n}$, $E_\tau \in \mathbb{R}^{n_\tau \times p_\tau}$ (one block per leaf), entries iid $U[0, 1]$	exact, rank k $(U_\tau = X(I_\tau, :),$ $V_\tau = Y(:, J_\tau)^T,$ $R = W = [I; I], B = I)$	$10^3 - 7 \times 10^3$, grows $\propto n$
F2	random HSS: D_τ iid $N(0, 1)/\sqrt{p_\tau}$; U, V, R, W with orthonormal columns (QR of Gaussian matrices); B iid $N(0, 1)/\sqrt{k}$; real or complex	exact, rank k	13–79
F2 $_\alpha$	F2 with graded columns: $H \text{diag}(10^{-\alpha u_j})$, u_j iid $U[0, 1]$, $\alpha \geq 0$	exact, same ranks as F2	$9.3 - 2 \times 10^{13}$
F3	$A_{ij} = 1/(x_i - y_j)$, $x_i = i$ ($i \leq M$), three points $y = x_i + q/4$ ($q = 1, 2, 3$) in each gap; $n = 3(M - 1)$	tolerance 10^{-10}	≈ 1.9
F4	$y_i = \sum_j g(si + w - j) x_j$: Gaussian (width σ) or Ricker wavelet g , truncated at $ d \leq w = \lceil 4\sigma \rceil$; stride s ; “valid” outputs only	exact (banded)	$3.0 - 10^8$, grows with σ
F5	$C = AF^*$ with $A_{jk} = e^{2\pi i k x_j}$, $k = -N/2, \dots, N/2 - 1$, jittered x_j , F the unitary DFT: $C_{jl} = D_N(x_j - l/N)/\sqrt{N}$, D_N the Dirichlet kernel; complex	tolerance 10^{-12}	≈ 2.0
F6	$C = F_m T D^* F_n^*$ with $T \in \mathbb{R}^{m \times 2m}$ random Toeplitz, F_m, F_n unitary DFTs, D a unit-modulus diagonal	tolerance 10^{-12}	≈ 7.4

Table 1: Test families. All are wide ($m < n$) and of full row rank. κ at the sizes of Table 2 (F2 $_\alpha$ and F4: over the sweeps of Section 7.4).

Notes on the families. F1 has a rank- k part with all-positive entries. Its largest singular value is therefore about $\frac{k}{4}\sqrt{mn}$, far above the rest of the spectrum, so $\kappa(\text{F1})$ grows linearly with n , and so does the error at large n (Section 7.5). F2 is the generic case: fixed rank and a small condition number. Scaling its columns (F2 $_\alpha$) keeps it exactly HSS with the same ranks and raises κ steadily with α (roughly like $10^{\alpha/2}$), which makes κ a parameter. F3 is a smooth kernel matrix; interlacing

the two point sets keeps $\kappa \approx 2$ at every size. F4 is a decimated convolution, as in deconvolution with a “valid” boundary or a stride: it is banded and therefore exactly HSS, and its condition number is set by the blur width σ . We use it to compare the direct solver with iterative methods as κ grows (Section 7.4). F5 and F6 come from matrices that are not HSS themselves: a nonuniform Fourier matrix, as in interpolating a band-limited signal from irregular samples, and a random Toeplitz matrix, whose off-diagonal blocks have full rank. Unitary DFTs turn both into Cauchy-like matrices, which are HSS [3]. By Lemma 4.2, a unitary change of variables maps minimum-norm solutions to minimum-norm solutions, so the solver applies to the transformed system unchanged.

7.3 Correctness at moderate size

Table 2 runs every family at a size where the dense references are affordable (`mp_exp_correctness.m`). In every case the residual is at most 7.7×10^{-14} and the solver error is at most $14 \kappa(H)u$; the largest ratios occur for $\kappa(H) < 10$, where the solver error is at most 4.2×10^{-15} . The null-space fraction, the part of x that a minimum-norm solution must not have, is of the same size as the solver error; a method that solved $Hx = b$ but drifted from the minimum-norm solution would show a null-space fraction far above the residual. The error against the original matrix adds the compression error: for F3, F5 and F6 it is 1.0×10^{-12} – 6.7×10^{-11} , of the size of the compression tolerance (10^{-10} for F3, 10^{-12} for F5 and F6), as the $2\kappa\epsilon$ bound of Section 4 predicts; for F1, F2 and F4, which are exactly HSS, it agrees with the solver error. The cases F2 $m/n = 0.9$, F5 and F6 violate the slack condition of Assumption 1 and are as accurate as the others.

case	m	n	L	rank	$\kappa(H)$	As. 1	residual	solver error	null frac.	vs. A
F1	320	480	4	3	1.3×10^3	yes	5.9×10^{-15}	2.3×10^{-14}	1.6×10^{-14}	2.3×10^{-14}
F1	2048	4096	7	3	7.4×10^3	yes	2.8×10^{-14}	1.9×10^{-13}	1.7×10^{-13}	1.9×10^{-13}
F2	1024	2048	5	10	14	yes	1.4×10^{-15}	3.6×10^{-15}	3.2×10^{-15}	3.6×10^{-15}
F2 complex	1024	2048	5	10	13	yes	1.5×10^{-15}	5.2×10^{-15}	4.6×10^{-15}	5.2×10^{-15}
F2 $m/n = 0.9$	1800	2000	5	8	79	no	2.5×10^{-15}	1.4×10^{-14}	1.2×10^{-14}	1.4×10^{-14}
F3	1024	3069	5	32	1.9	yes	1.1×10^{-15}	2.2×10^{-15}	2.2×10^{-15}	6.7×10^{-11}
F4 Gaussian, $\sigma = 2, s = 2$	2040	4096	5	10	98	yes	6.6×10^{-15}	1.0×10^{-14}	8.7×10^{-15}	1.0×10^{-14}
F4 Ricker, $\sigma = 3, s = 2$	2036	4096	5	15	2.2×10^3	yes	7.7×10^{-14}	1.9×10^{-13}	1.9×10^{-13}	2.0×10^{-13}
F4 Gaussian, $\sigma = 3, s = 3$	1358	4096	5	10	98	yes	5.1×10^{-15}	1.0×10^{-14}	9.3×10^{-15}	1.0×10^{-14}
F5	1536	2048	5	36	2.0	no	1.2×10^{-15}	3.1×10^{-15}	3.1×10^{-15}	1.0×10^{-12}
F6	1024	2048	5	62	7.4	no	2.1×10^{-15}	4.2×10^{-15}	3.7×10^{-15}	1.4×10^{-12}

Table 2: Accuracy at moderate size. “rank”: largest generator rank; L : depth; “As. 1”: whether every leaf satisfies the slack condition. “vs. A ”: error against the minimum-norm solution of the original matrix (for F6, in the Toeplitz variables, $x = D^* F_n^* y$).

7.4 Accuracy as the condition number grows

Figure 2 sweeps $\kappa(H)$ in the two families where it is a parameter, $F2_\alpha$ (depth 3) and F4 with stride 1 and growing σ (depth 4), and compares the ULV solve with the normal equations (Cholesky factorization of HH^*) and with CGNE (conjugate gradients on HH^* with HSS products, relative-residual tolerance 10^{-14} , at most 2000 iterations). All are measured against the dense QR solution (`mp_exp_conditioning.m`). For an ill-conditioned problem that reference is itself accurate only to about κu ($u \approx 1.1 \times 10^{-16}$), so differences below κu cannot be resolved: a curve at or below the κu line means the method is as accurate as dense QR.

The ULV solution stays within $3 \kappa u$ of the dense QR solution over the whole range, up to $\kappa \approx 1.8 \times 10^{13}$ for $F2_\alpha$ and 1.1×10^8 for F4: it is as accurate as dense QR. The normal equations follow $\kappa^2 u$, as squaring the condition number predicts, and their Cholesky factorization fails

from $\kappa \approx 1.7 \times 10^9$ (F2 $_{\alpha}$) and 1.1×10^8 (F4) on. CGNE needs 99 iterations already at $\kappa \approx 10$; from $\kappa \approx 9.8 \times 10^3$ (F2 $_{\alpha}$) and 6.1×10^2 (F4) on it does not converge within 2000 iterations and its error exceeds 10^{-6} . This is the regime that needs a direct solver. The backward error $\|b - Hx\|/(\|H\|\|x\| + \|b\|)$, which needs no reference, is the smallest relative perturbation of H and b for which x solves the system exactly; it stays below 3.5×10^{-16} for the ULV solution throughout (dense QR: 2.2×10^{-16}) and grows to 1.0×10^{-8} for the normal equations.

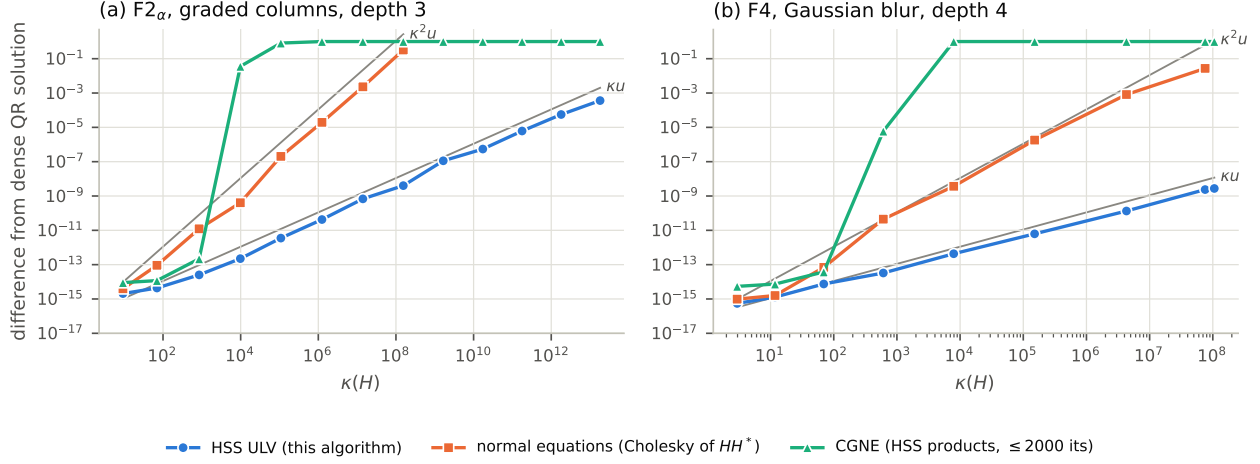


Figure 2: Relative difference from the dense QR minimum-norm solution as $\kappa(H)$ grows. (a) F2 $_{\alpha}$, 128 $\times$ 256, depth 3; (b) F4, Gaussian blur, stride 1, $\approx 290 \times 300$, depth 4. Grey lines: κu and $\kappa^2 u$. Missing normal-equation points: the Cholesky factorization of HH^* failed. CGNE values of 1: no iterate had a smaller residual than the starting guess 0, which is returned.

7.5 Accuracy at large n

With manufactured solutions the solver can be checked at sizes where no dense matrix fits (Figure 3, `mp_exp_scaling.m`, the same run as Figure 1). For F2–F5 the error stays between 1.7×10^{-15} and 5.6×10^{-15} for every n up to 2^{17} . For F1 it grows from 7.8×10^{-14} to 5.6×10^{-12} , in step with $\kappa(\text{F1}) \propto n$ (Section 7.2): this is the conditioning of the test matrix, not error growth in the solver. Residuals are at most 2.8×10^{-15} .

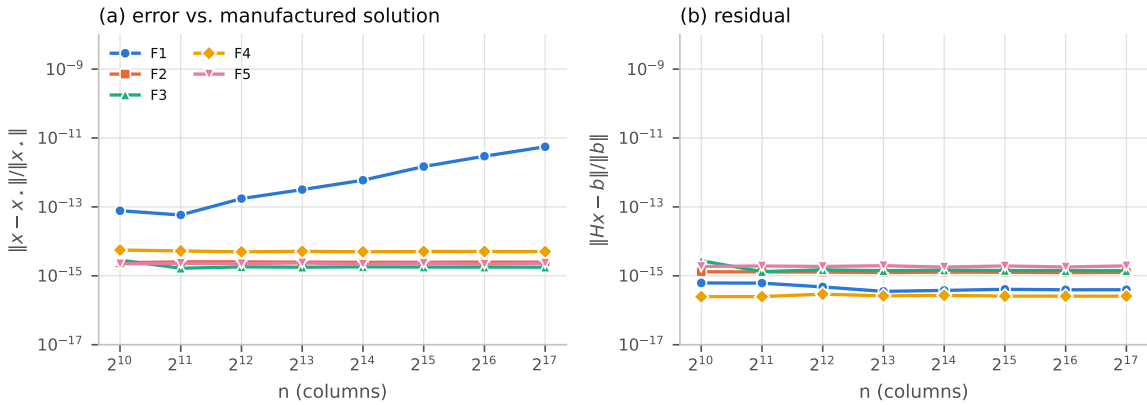


Figure 3: Error against the manufactured minimum-norm solution as n grows. No dense matrix is formed.

8 Open Problems

1. **Floating-point analysis.** The experiments are consistent with backward stability, but there is no proof. A natural route is the backward-error analysis of ULV for square HSS matrices, extended by the size reduction.
2. **Leaf size.** The cost model of Section 5 is smallest for leaves of about r rows; choosing the leaf size from the ranks, instead of fixing it, should lower the constant.
3. **Rank-deficient H .** The reduction is exact only for full row rank. Extending it to the minimum-norm least-squares solution would cover inconsistent and rank-deficient systems.
4. **Higher dimensions.** For 2-D and 3-D problems the HSS ranks grow with the size of the interfaces between clusters; formats with strong admissibility ($\mathcal{H}^2$, recursive skeletonization) are needed, and the reduction of Section 3 should carry over block by block.

References

- [1] S. Chandrasekaran, M. Gu, T. Pals, A fast ULV decomposition solver for hierarchically semiseparable representations, *SIAM J. Matrix Anal. Appl.* 28 (2006).
- [2] J. Demmel, N. J. Higham, Improved error bounds for underdetermined system solvers, *SIAM J. Matrix Anal. Appl.* 14 (1993).
- [3] I. Gohberg, T. Kailath, V. Olshevsky, Fast Gaussian elimination with partial pivoting for matrices with displacement structure, *Math. Comp.* 64 (1995).
- [4] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.